sajibkhansk

SQL ANALYTICS HANDBOOK

Business & Operational Analytics with SQL

Thirty-three production-style queries on a real e-commerce database, every result run and verified. With ten practice questions.

REVENUE & SALES | CUSTOMER BEHAVIOUR | OPERATIONS

TechMart Electronics dataset • MySQL 8.4 • 432 orders, Jan to Sep 2026

BEFORE YOU START

The TechMart Dataset

Eight tables, 432 orders, nine months of trading. Every figure in this handbook was produced by running the query shown against this database.

TABLES AND ROW COUNTS

TABLE	ROWS	ROLE
regions	4	Dimension
statuses	9	Dimension
employees	7	Sales reps
customers	25	Corporate / SMB / Consumer
products	20	Six categories
orders	432	One row per order
order_items	1,380	Line items, holds price and discount
order_status_log	2,148	Every status change, timestamped

STATUS REFERENCE

ID	STATUS	MEANING
1	Pending	Placed, not yet confirmed
2	Confirmed	Accepted
3	Processing	Being picked and packed
4	Shipped	With the carrier
5	Delivered	Completed successfully
6	Cancelled	Killed before shipping
7	Return Requested	Customer asked to return
8	Returned	Goods came back
9	Refunded	Money returned

TWO CONVENTIONS USED THROUGHOUT

1. Line revenue is `quantity × unit_price × (1 - discount)`. The discount lives on `order_items`, not on the order, so it must be applied per line. **2. Revenue queries filter** `current_status_id NOT IN (6,7,8,9)` to exclude cancelled and returned orders. Operational queries deliberately do not, because you cannot measure a cancellation rate on data with cancellations removed.

VERIFIED TOTALS YOU CAN CHECK YOUR OWN WORK AGAINST

Net revenue 3,227,602.05 • 366 orders delivered (84.7%) • 48 cancelled (11.1%) • 35 refunded (8.1%) • average delivery 5.5 days.

BEFORE YOU START

How to Read This Handbook

Each query gets one page with the same four parts, so you can skim for the pattern you need and drop straight into the SQL.

1. Problem
The business question in one sentence, phrased the way a stakeholder would actually ask it.

2. SQL
The full query, runnable as printed against the TechMart schema. Nothing elided.

3. Result
Real output from running it. Where a table is trimmed to fit, the caption says so.

4. What it tells you
The finding, and the SQL technique worth stealing for your own work.

Three queries were changed, and why

QUERY	PROBLEM FOUND WHEN RUN	FIX
Discount sensitivity	Grouped by an aggregate expression, which MySQL rejects	Compute the discount tier per order in a CTE, then group
Churn risk	Nested `LAG()` inside `AVG()`, which is not legal	Compute gaps in a CTE, average them in the next
SLA breach report	Returned zero rows: no order ships later than 3 days	Reframed as an SLA compliance and at-risk view

WHY THAT MATTERS MORE THAN IT LOOKS

All three read fine on the page. They only failed when executed. A query you have not run is a guess, and a result table you have not checked for meaning is worse than none at all, because it looks authoritative.

RUNNING THESE YOURSELF

Load the schema, then use `techmart_db`. Everything here is standard MySQL 8: CTEs, window functions and `NOT IN` subqueries all work natively with no extensions.

CHAPTER 1

Revenue & Sales Analytics

Ten queries that answer the questions a founder or sales lead asks first: how much are we making, is it growing, who is buying, and what is actually profitable.

`Monthly revenue`
Baseline trend: orders, units and revenue per month

`MoM growth`
LAG to compare each month against the last

`Cumulative revenue`
SUM OVER for a running year-to-date total

`Top customers`
RANK to find who carries the business

`Category margin`
Revenue is not profit; margin separates them

`Market basket`
Self-join to find products bought together

THE REVENUE FORMULA USED EVERYWHERE IN THIS HANDBOOK

Line revenue is always `quantity × unit_price × (1 - discount)`, and every revenue query filters `current_status_id NOT IN (6,7,8,9)` to exclude cancelled, returned and refunded orders. Apply that filter consistently or your numbers will not reconcile.

SCALE OF THIS DATASET

432 orders across 25 customers and 20 products, January to September 2026. Total net revenue is **3,227,602.05**. Small enough to verify by hand, large enough for the patterns to be real.

REVENUE & SALES

Monthly Revenue Summary

Problem: How much did we sell each month, in orders, units and net revenue?

```sql
SELECT
    DATE_FORMAT(o.order_date, '%Y-%m')                              AS month,
    COUNT(DISTINCT o.id)                                            AS total_orders,
    SUM(oi.quantity)                                                AS units_sold,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue,
    ROUND(AVG(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS avg_line_value
FROM orders o
JOIN order_items oi ON o.id = oi.order_id
WHERE o.current_status_id NOT IN (6, 7, 8, 9)
GROUP BY DATE_FORMAT(o.order_date, '%Y-%m')
ORDER BY month;
```

RESULT (ALL 9 MONTHS)

MONTH	TOTAL_ORDERS	UNITS_SOLD	REVENUE	AVG_LINE_VALUE
2026-01	35	492	348,626.45	3,085.19
2026-02	38	602	309,886.80	2,582.39
2026-03	34	589	323,808.00	2,840.42
2026-04	42	593	372,597.45	2,910.92
2026-05	43	583	378,554.25	2,934.53
2026-06	41	630	377,738.10	2,717.54
2026-07	38	558	355,937.40	2,870.46
2026-08	38	602	324,594.20	2,660.61
2026-09	40	647	435,859.40	3,301.97

WHAT IT TELLS YOU

September is the strongest month at 435,859 on only 40 orders. Revenue is not tracking order count, it is tracking basket value: avg_line_value climbs to 3,301 in September against 2,582 in February. Bigger baskets, not more orders, drove the best month.

REVENUE & SALES

Month-over-Month Growth

Problem: Is revenue growing or shrinking, and by how much each month?

```sql
WITH monthly AS (
    SELECT DATE_FORMAT(o.order_date, '%Y-%m')                          AS month,
           ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue
    FROM orders o
    JOIN order_items oi ON o.id = oi.order_id
    WHERE o.current_status_id NOT IN (6, 7, 8, 9)
    GROUP BY DATE_FORMAT(o.order_date, '%Y-%m')
)
SELECT
    month, revenue,
    LAG(revenue) OVER (ORDER BY month) AS prev_month_revenue,
    ROUND((revenue - LAG(revenue) OVER (ORDER BY month)) * 100.0
          / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 1) AS mom_growth_pct
FROM monthly
ORDER BY month;
```

RESULT

MONTH	REVENUE	PREV_MONTH_REVENUE	MOM_GROWTH_PCT
2026-01	348,626.45	*NULL*	*NULL*
2026-02	309,886.80	348,626.45	-11.1
2026-03	323,808.00	309,886.80	4.5
2026-04	372,597.45	323,808.00	15.1
2026-05	378,554.25	372,597.45	1.6
2026-06	377,738.10	378,554.25	-0.2
2026-07	355,937.40	377,738.10	-5.8
2026-08	324,594.20	355,937.40	-8.8
2026-09	435,859.40	324,594.20	34.3

January is NULL because there is no prior month to compare against.

WHAT IT TELLS YOU

The business is volatile, not steadily growing. Three consecutive declines (June to August) are followed by a 34.3% jump in September. A single month's growth number would badly mislead here; the trend only makes sense across the full series.

REVENUE & SALES

How LAG Powers Growth Metrics

1. The **CTE aggregates first.** You cannot compare months until months exist as rows. `monthly` collapses 1,380 line items into 9 rows, one per month.
2. **LAG(revenue) OVER (ORDER BY month)** reaches one row backward in the ordered result and returns that row's revenue. No self-join needed.
3. The **first row has nothing behind it**, so LAG returns NULL. That NULL then propagates through the arithmetic, which is why January's growth is NULL rather than zero. Reporting it as 0% would be wrong.
4. **NULLIF(prev, 0)** guards against divide-by-zero. If a month had zero revenue, NULLIF turns the denominator into NULL and the result becomes NULL instead of raising an error.

WITHOUT A CTE

You would need a self-join of the aggregate against itself on a month offset. Correct, but far harder to read and to change later.

WITH CTE PLUS LAG

Aggregate once, compare in a single pass. Swapping LAG for LEAD gives next-month comparison with no other change.

REUSE THIS SHAPE

Aggregate in a CTE, then apply a window function on top. That two-step pattern covers most period-over-period reporting: week-over-week, year-over-year, and cohort deltas all use it.

COMMON MISTAKE

Writing `LAG(SUM(revenue))` directly in the outer query. Window functions run **after** aggregation, so mixing them in one level either errors or silently gives the wrong frame. Keep the aggregation in its own CTE.

REVENUE & SALES

Cumulative Revenue (Running Total)

Problem: What is our year-to-date revenue after each month, and what share did each month contribute?

```sql
WITH monthly AS (
    SELECT DATE_FORMAT(o.order_date, '%Y-%m')                          AS month,
           ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS monthly_revenue
    FROM orders o
    JOIN order_items oi ON o.id = oi.order_id
    WHERE o.current_status_id NOT IN (6, 7, 8, 9)
    GROUP BY DATE_FORMAT(o.order_date, '%Y-%m')
)
SELECT
    month, monthly_revenue,
    SUM(monthly_revenue) OVER (ORDER BY month
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumulative_revenue,
    ROUND(monthly_revenue * 100.0 / SUM(monthly_revenue) OVER (), 1) AS pct_of_ytd
FROM monthly
ORDER BY month;
```

RESULT

MONTH	MONTHLY_REVENUE	CUMULATIVE_REVENUE	PCT_OF_YTD
2026-01	348,626.45	348,626.45	10.8
2026-02	309,886.80	658,513.25	9.6
2026-03	323,808.00	982,321.25	10.0
2026-04	372,597.45	1,354,918.70	11.5
2026-05	378,554.25	1,733,472.95	11.7
2026-06	377,738.10	2,111,211.05	11.7
2026-07	355,937.40	2,467,148.45	11.0
2026-08	324,594.20	2,791,742.65	10.1
2026-09	435,859.40	3,227,602.05	13.5

WHAT IT TELLS YOU

Two windows in one query doing different jobs. The framed window accumulates month by month; the bare `OVER ()` with no ORDER BY spans every row at once, giving the grand total used as the denominator for pct_of_ytd. Same function, entirely different frame.

REVENUE & SALES

Top Customers by Revenue

Problem: Which customers generate the most revenue, and how concentrated is that at the top?

```sql
SELECT
    c.full_name, c.segment, c.city,
    COUNT(DISTINCT o.id)                                        AS orders,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS total_revenue,
    RANK() OVER (ORDER BY SUM(oi.quantity * oi.unit_price
                           * (1 - oi.discount)) DESC)           AS revenue_rank
FROM customers c
JOIN orders o       ON c.id = o.customer_id
JOIN order_items oi ON o.id = oi.order_id
WHERE o.current_status_id NOT IN (6, 7, 8, 9)
GROUP BY c.id, c.full_name, c.segment, c.city
ORDER BY total_revenue DESC
LIMIT 10;
```

RESULT (TOP 10 OF 25 CUSTOMERS)

FULL_NAME	SEGMENT	CITY	ORDERS	TOTAL_REVENUE	REVENUE_RANK
Vertex Systems	Corporate	Boston	38	590,866.40	1
Acme Corp	Corporate	Chicago	37	579,033.55	2
GreenPath Inc	Corporate	Phoenix	36	521,289.75	3
Apex Dynamics	Corporate	New York	20	390,330.45	4
TechNova Ltd	Corporate	San Jose	17	256,748.70	5
DataBridge Co	Corporate	Austin	16	169,750.65	6
PineCrest Digital	SMB	Denver	18	131,822.55	7
BlueSky LLC	SMB	Miami	17	107,393.40	8
Orbit Creative	SMB	Nashville	11	86,185.95	9
Redstone Partners	SMB	Atlanta	12	54,245.65	10

WHAT IT TELLS YOU

The top six are all Corporate and together account for roughly 2.5M of 3.2M total revenue. Apex Dynamics is the standout: rank 4 on only 20 orders, half of what Vertex placed. Fewer, much larger orders. Losing any one of the top three would remove over 15% of revenue.

REVENUE & SALES

Category Performance: Revenue vs Margin

Problem: Which categories bring in revenue, and which actually make money after cost?

```sql
SELECT
    p.category,
    COUNT(DISTINCT o.id)                                        AS orders,
    SUM(oi.quantity)                                            AS units_sold,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue,
    ROUND(SUM(oi.quantity * (oi.unit_price * (1 - oi.discount)
                           - p.cost_price)), 2)                 AS gross_profit,
    ROUND(SUM(oi.quantity * (oi.unit_price * (1 - oi.discount) - p.cost_price))
          * 100.0 / NULLIF(SUM(oi.quantity * oi.unit_price
                               * (1 - oi.discount)), 0), 1)     AS margin_pct
FROM products p
JOIN order_items oi ON p.id = oi.product_id
JOIN orders o       ON oi.order_id = o.id
WHERE o.current_status_id NOT IN (6, 7, 8, 9)
GROUP BY p.category
ORDER BY revenue DESC;
```

RESULT (ALL 6 CATEGORIES)

CATEGORY	ORDERS	UNITS_SOLD	REVENUE	GROSS_PROFIT	MARGIN_PCT
Laptops	190	1,195	1,887,938.80	361,298.80	19.1
Monitors	197	1,176	766,978.75	229,888.75	30.0
Accessories	287	2,683	379,449.15	156,712.15	41.3
Tablets	62	157	137,835.40	40,095.40	29.1
Phones	24	37	38,664.00	11,124.00	28.8
Audio	34	48	16,735.95	7,050.95	42.1

WHAT IT TELLS YOU

This is the single most important table in the chapter. Laptops generate 58% of revenue but at the **worst margin in the catalogue, 19.1%**. Accessories earn 41.3% on a fifth of the revenue. Ranked by revenue Laptops win; ranked by profit-per-sales-dollar they come last. Never judge a category on revenue alone.

REVENUE & SALES

Best-Selling Products

Problem: Which individual products drive revenue and profit?

```sql
SELECT
    p.product_name, p.category,
    SUM(oi.quantity)                                          AS units_sold,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue,
    ROUND(SUM(oi.quantity * (oi.unit_price * (1 - oi.discount)
                           - p.cost_price)), 2)               AS profit,
    RANK() OVER (ORDER BY SUM(oi.quantity * oi.unit_price
                            * (1 - oi.discount)) DESC)        AS rank_by_revenue
FROM products p
JOIN order_items oi ON p.id = oi.product_id
JOIN orders o       ON oi.order_id = o.id
WHERE o.current_status_id NOT IN (6, 7, 8, 9)
GROUP BY p.id, p.product_name, p.category
ORDER BY revenue DESC
LIMIT 12;
```

RESULT (TOP 12 OF 20 PRODUCTS)

PRODUCT_NAME	CATEGORY	UNITS_SOLD	REVENUE	PROFIT	RANK_BY_REVENUE
MacBook Pro 14	Laptops	422	751,624.00	139,724.00	1
Dell XPS 15	Laptops	415	569,180.05	112,680.05	2
ASUS ROG Zephyrus G14	Laptops	358	567,134.75	108,894.75	3
Samsung 49 Ultrawide	Monitors	426	453,281.95	121,001.95	4
Dell 32 Curved Monitor	Monitors	359	175,981.95	61,101.95	5
CalDigit TS4 Dock	Accessories	457	140,507.40	49,107.40	6
LG 27 4K Monitor	Monitors	391	137,714.85	47,784.85	7
Webcam Logitech 4K	Accessories	442	78,883.60	32,473.60	8
Microsoft Surface Pro 10	Tablets	54	67,418.10	18,818.10	9
Elgato Stream Deck	Accessories	466	60,643.00	25,693.00	10
Keychron K2 Keyboard	Accessories	417	39,959.40	17,024.40	11
Logitech MX Master 3	Accessories	451	39,694.05	20,752.05	12

WHAT IT TELLS YOU

Units and revenue disagree sharply. Elgato Stream Deck sells the most units of any product (466) yet ranks 10th on revenue. Samsung 49 Ultrawide turns 426 units into 453K and out-profits Dell XPS 15 despite lower revenue. Volume, revenue and profit are three different leaderboards.

REVENUE & SALES

Revenue Concentration by Segment

Problem: What share of revenue does each customer segment produce, and what does a typical order look like in each?

```sql
WITH seg_rev AS (
    SELECT c.segment,
           ROUND(SUM(oi.quantity * oi.unit_price
                     * (1 - oi.discount)), 2) AS revenue
    FROM customers c
    JOIN orders o       ON c.id = o.customer_id
    JOIN order_items oi ON o.id = oi.order_id
    WHERE o.current_status_id NOT IN (6,7,8,9)
    GROUP BY c.segment
)
SELECT segment, revenue,
       ROUND(revenue * 100.0
             / SUM(revenue) OVER (), 1) AS pct_of_total
FROM seg_rev ORDER BY revenue DESC;
```

REVENUE SHARE

SEGMENT	REVENUE	PCT_OF_TOTAL
Corporate	2,508,019.50	77.7
SMB	503,152.00	15.6
Consumer	216,430.55	6.7

```sql
SELECT c.segment,
    COUNT(DISTINCT o.id)  AS total_orders,
    COUNT(DISTINCT c.id)  AS unique_customers,
    ROUND(AVG(t.order_value), 2) AS avg_order_value,
    ROUND(MAX(t.order_value), 2) AS max_order_value,
    ROUND(MIN(t.order_value), 2) AS min_order_value
FROM customers c
JOIN orders o ON c.id = o.customer_id
JOIN (
    SELECT order_id,
           SUM(quantity * unit_price
               * (1 - discount)) AS order_value
    FROM order_items GROUP BY order_id
) t ON o.id = t.order_id
WHERE o.current_status_id NOT IN (6,7,8,9)
GROUP BY c.segment ORDER BY avg_order_value DESC;
```

ORDER VALUE BY SEGMENT

SEGMENT	ORDERS	CUSTS	AVG_VALUE	MAX
Corporate	164	6	15,292.80	37,723.60
SMB	87	7	5,783.36	18,253.85
Consumer	98	12	2,208.48	8,059.20

THE CONCENTRATION RISK IN ONE LINE

Six Corporate customers produce 77.7% of all revenue. Twelve Consumer customers, twice as many people, produce 6.7%. A Corporate order averages 15,292 against the Consumer 2,208, a **6.9x gap**. That shapes everything downstream: who sales calls, which SLA matters, and whose churn would actually hurt.

REVENUE & SALES

Employee Sales Leaderboard

Problem: Who is closing the most revenue, and what commission has each rep earned at 3%?

```sql
SELECT
    e.full_name, e.role, r.region_name,
    COUNT(DISTINCT o.id)                                        AS orders_closed,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS total_revenue,
    ROUND(SUM(oi.quantity * oi.unit_price
             * (1 - oi.discount)) * 0.03, 2)                    AS commission_est,
    RANK() OVER (ORDER BY SUM(oi.quantity * oi.unit_price
                           * (1 - oi.discount)) DESC)           AS rank_overall
FROM employees e
JOIN regions r      ON e.region_id = r.id
JOIN orders o       ON e.id = o.employee_id
JOIN order_items oi ON o.id = oi.order_id
WHERE o.current_status_id NOT IN (6, 7, 8, 9)
GROUP BY e.id, e.full_name, e.role, r.region_name
ORDER BY total_revenue DESC;
```

RESULT (ALL 7 REPS)

FULL_NAME	ROLE	REGION_NAME	ORDERS	TOTAL_REVENUE	COMMISSION_EST	RANK
Marcus Webb	Sales Rep	South	54	552,834.55	16,585.04	1
Priya Sharma	Senior Rep	East	56	519,384.15	15,581.52	2
Lena Fischer	Senior Rep	North	50	498,343.80	14,950.31	3
James Caldwell	Sales Rep	West	48	454,450.20	13,633.51	4
David Okonkwo	Sales Rep	South	50	444,604.30	13,338.13	5
Sarah Mitchell	Sales Rep	North	50	384,017.80	11,520.53	6
Aisha Rahman	Sales Rep	East	41	373,967.25	11,219.02	7

Commission is estimated at a flat 3% of net revenue.

WHAT IT TELLS YOU

Marcus Webb tops the table on fewer orders than Priya Sharma (54 against 56), so he is winning on deal size, not activity. The spread from first to last is only 1.48x, which is healthy: no single rep is carrying the team, and no one is badly adrift.

REVENUE & SALES

Products Frequently Bought Together

Problem: Which product pairs appear in the same order most often? This is the basis of a recommendation engine.

```sql
SELECT
    p1.product_name   AS product_a,
    p2.product_name   AS product_b,
    COUNT(*)          AS times_together
FROM order_items a
JOIN order_items b
    ON a.order_id = b.order_id
   AND a.product_id < b.product_id       -- avoids (X,Y) and (Y,X) duplicates
JOIN products p1 ON a.product_id = p1.id
JOIN products p2 ON b.product_id = p2.id
GROUP BY p1.product_name, p2.product_name
HAVING COUNT(*) >= 3
ORDER BY times_together DESC
LIMIT 12;
```

RESULT (TOP 12 PAIRS)

PRODUCT_A	PRODUCT_B	TIMES_TOGETHER
Anker USB-C Hub	CalDigit TS4 Dock	33
Logitech MX Master 3	CalDigit TS4 Dock	32
Logitech MX Master 3	Anker USB-C Hub	31
Keychron K2 Keyboard	Anker USB-C Hub	31
LG 27 4K Monitor	Keychron K2 Keyboard	30
LG 27 4K Monitor	CalDigit TS4 Dock	30
Logitech MX Master 3	Samsung 49 Ultrawide	29
Webcam Logitech 4K	Samsung 49 Ultrawide	29
CalDigit TS4 Dock	Webcam Logitech 4K	29
Dell XPS 15	Logitech MX Master 3	29
MacBook Pro 14	Logitech MX Master 3	29
Dell XPS 15	Anker USB-C Hub	29

WHAT IT TELLS YOU

The trick is `a.product_id < b.product_id`. A plain self-join on order_id would match every row to itself and return each pair twice in both orders. The less-than comparison keeps exactly one copy of each pair and drops self-matches, in one condition. The result: accessories cluster together, and every laptop pairs with a mouse or a hub, which is a ready-made bundle offer.

CHAPTER 1 REFERENCE

Revenue Patterns Cheat Sheet

GOAL	PATTERN	KEY CLAUSE
Revenue per period	Aggregate + GROUP BY date bucket	`DATE_FORMAT(d,'%Y-%m')`
Compare to prior period	CTE, then LAG over the aggregate	`LAG(x) OVER (ORDER BY month)`
Running / YTD total	Framed window sum	`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`
Share of grand total	Bare window with no ORDER BY	`x * 100.0 / SUM(x) OVER ()`
Leaderboard with ties	RANK over the aggregate	`RANK() OVER (ORDER BY SUM(...) DESC)`
Top N per group	ROW_NUMBER partitioned, filter outside	`PARTITION BY segment` then `WHERE rn <= 3`
Order-level metric	Pre-aggregate to one row per order, then join	derived table on `order_id`
Profit and margin	Subtract cost inside the SUM	`SUM(qty * (price - cost))`
Safe percentage	Guard the denominator	`NULLIF(denominator, 0)`
Pairs from one table	Self-join with a less-than condition	`a.product_id < b.product_id`

THE MISTAKE THAT BREAKS REVENUE REPORTS

Averaging an average. `AVG(order_value)` across line items is not the average order value, because orders with more lines get counted more often. Collapse to one row per order first, then average. The AOV query on the segment page does exactly that with a derived table.

RECONCILIATION HABIT

Every revenue number in this chapter rolls up to 3,227,602.05. If you add a filter or a join and that total moves, the join multiplied rows. Check the total before trusting any breakdown built on top of it.

CHAPTER 2

Customer Behaviour

Revenue tells you what happened. Behaviour tells you what is about to happen: who is coming back, who is drifting away, and what they buy on the way in.

Frequency One-timers vs loyalists	**Interval** Days between repeat orders	**Lifetime value** Total worth per customer
Cohorts Retention by signup month	**Churn risk** Quiet relative to their own habit	**Entry product** What they buy first

WHY RECENCY IS MEASURED AGAINST THE DATA, NOT TODAY

Two queries in this chapter compare a last-order date to a reference date. Using `CURDATE()` on a static dataset produces **negative day counts** for any order dated after today. Both queries here use `(SELECT MAX(order_date) FROM orders)` as the as-of date instead, which is the standard practice for a fixed extract and keeps the numbers sane.

A CAVEAT ON SAMPLE SIZE

This dataset has 25 customers. That is enough to demonstrate every technique correctly, but too few for the path and cohort analyses to carry statistical weight. Where that matters, the page says so rather than dressing up a thin result.

CUSTOMER BEHAVIOUR

Purchase Frequency Distribution

Problem: Are most customers one-time buyers, or do they come back?

```sql
SELECT frequency_bucket,
       COUNT(*)                                          AS customers,
       ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 1) AS pct_of_customers
FROM (
    SELECT c.id, COUNT(DISTINCT o.id) AS order_count,
        CASE WHEN COUNT(DISTINCT o.id) = 1             THEN '1 order (one-time)'
             WHEN COUNT(DISTINCT o.id) BETWEEN 2 AND 3 THEN '2-3 orders'
             WHEN COUNT(DISTINCT o.id) BETWEEN 4 AND 6 THEN '4-6 orders'
             ELSE '7+ orders (loyal)' END AS frequency_bucket
    FROM customers c
    JOIN orders o ON c.id = o.customer_id
    WHERE o.current_status_id NOT IN (6)
    GROUP BY c.id
) buckets
GROUP BY frequency_bucket
ORDER BY MIN(order_count);
```

RESULT

FREQUENCY_BUCKET	CUSTOMERS	PCT_OF_CUSTOMERS
2-3 orders	1	4.0
4-6 orders	3	12.0
7+ orders (loyal)	21	84.0

WHAT IT TELLS YOU

There is no one-time bucket at all: every customer in this dataset has ordered at least twice, and 84% have ordered seven or more times. This is a repeat-purchase business, which means retention work matters far more than acquisition. Note the bucketing runs in a subquery because you cannot GROUP BY a CASE that depends on an aggregate.

CUSTOMER BEHAVIOUR

Days Between Purchases

Problem: How long does each customer typically wait before ordering again?

```sql
WITH ordered AS (
    SELECT o.customer_id, o.order_date,
           LAG(o.order_date) OVER (PARTITION BY o.customer_id
                                   ORDER BY o.order_date) AS prev_order_date
    FROM orders o
    WHERE o.current_status_id NOT IN (6)
)
SELECT c.full_name, c.segment,
       COUNT(*)                                          AS repeat_purchases,
       ROUND(AVG(DATEDIFF(order_date, prev_order_date)), 0) AS avg_days_between,
       MIN(DATEDIFF(order_date, prev_order_date))        AS fastest_days,
       MAX(DATEDIFF(order_date, prev_order_date))        AS slowest_days
FROM ordered ord
JOIN customers c ON ord.customer_id = c.id
WHERE prev_order_date IS NOT NULL
GROUP BY c.id, c.full_name, c.segment
ORDER BY avg_days_between ASC;
```

RESULT (12 FASTEST-RETURNING CUSTOMERS)

FULL_NAME	SEGMENT	REPEATS	AVG_DAYS	FASTEST	SLOWEST
Vertex Systems	Corporate	39	6	0	20
Acme Corp	Corporate	38	7	0	29
GreenPath Inc	Corporate	37	7	0	29
Apex Dynamics	Corporate	22	12	0	29
TechNova Ltd	Corporate	18	12	0	45
Orbit Creative	SMB	14	13	0	45
Priya Mehta	Consumer	12	13	1	34
PineCrest Digital	SMB	18	14	1	42
BlueSky LLC	SMB	18	15	0	64
WaveLink Tech	SMB	17	15	1	47
DataBridge Co	Corporate	16	16	1	45
Jake Morrison	Consumer	10	17	3	43

WHERE prev_order_date IS NOT NULL drops each customer's first order, which has nothing to compare against.

WHAT IT TELLS YOU

Corporate accounts reorder every 6 to 7 days; everyone else sits at 13 to 17. A fastest_days of 0 means two orders on the same date, which is normal for corporate purchasing. These averages become the baseline for churn detection: quiet is only meaningful relative to a customer's own rhythm.

CUSTOMER BEHAVIOUR

Customer Lifetime Value

Problem: What is each customer worth in total, and how long have they been active?

```sql
SELECT c.full_name, c.segment, c.city,
    COUNT(DISTINCT o.id)                                        AS total_orders,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS lifetime_revenue,
    ROUND(AVG(t.order_value), 2)                                AS avg_order_value,
    DATEDIFF(MAX(o.order_date), MIN(o.order_date))              AS lifespan_days,
    MIN(o.order_date) AS first_order, MAX(o.order_date) AS last_order
FROM customers c
JOIN orders o       ON c.id = o.customer_id
JOIN order_items oi ON o.id = oi.order_id
JOIN (SELECT order_id, SUM(quantity * unit_price * (1 - discount)) AS order_value
      FROM order_items GROUP BY order_id) t ON o.id = t.order_id
WHERE o.current_status_id NOT IN (6)
GROUP BY c.id, c.full_name, c.segment, c.city
ORDER BY lifetime_revenue DESC;
```

RESULT (TOP 12 BY LIFETIME REVENUE)

FULL_NAME	SEGMENT	ORDERS	LIFETIME_REV	AVG_ORDER	LIFESPAN	FIRST_ORDER
Vertex Systems	Corporate	40	609,096.45	15,573.05	244	2026-01-07
Acme Corp	Corporate	39	603,520.90	16,214.21	263	2026-01-06
GreenPath Inc	Corporate	38	550,327.80	14,537.50	255	2026-01-14
Apex Dynamics	Corporate	23	445,993.00	19,657.71	254	2026-01-11
TechNova Ltd	Corporate	19	283,535.45	15,282.15	223	2026-02-12
DataBridge Co	Corporate	17	176,354.55	10,790.24	263	2026-01-02
PineCrest Digital	SMB	19	132,526.30	7,359.22	256	2026-01-05
Orbit Creative	SMB	15	116,831.70	7,964.56	185	2026-03-13
BlueSky LLC	SMB	19	113,739.30	6,285.69	270	2026-01-02
WaveLink Tech	SMB	18	83,962.55	5,135.58	253	2026-01-10
Redstone Partners	SMB	14	60,248.30	4,730.31	235	2026-01-01
SilverEdge Media	SMB	12	52,434.55	4,952.41	227	2026-02-12

WHAT IT TELLS YOU

Apex Dynamics has the highest average order value in the business at 19,657, beating Vertex Systems by 26%, yet sits 4th on lifetime revenue because it orders half as often. That is the CLV lever in plain sight: Apex does not need bigger baskets, it needs more frequent ones.

CUSTOMER BEHAVIOUR

New vs Returning Customers

Problem: Is growth coming from new customers or from existing ones buying again?

```sql
WITH first_orders AS (
    SELECT customer_id, MIN(order_date) AS first_order_date
    FROM orders WHERE current_status_id NOT IN (6)
    GROUP BY customer_id
)
SELECT DATE_FORMAT(o.order_date, '%Y-%m')  AS month,
    COUNT(DISTINCT o.customer_id)          AS total_customers,
    COUNT(DISTINCT CASE WHEN DATE_FORMAT(fo.first_order_date, '%Y-%m')
                          = DATE_FORMAT(o.order_date, '%Y-%m')
                THEN o.customer_id END) AS new_customers,
    COUNT(DISTINCT CASE WHEN DATE_FORMAT(fo.first_order_date, '%Y-%m')
                          < DATE_FORMAT(o.order_date, '%Y-%m')
                THEN o.customer_id END) AS returning_customers
FROM orders o
JOIN first_orders fo ON o.customer_id = fo.customer_id
WHERE o.current_status_id NOT IN (6)
GROUP BY DATE_FORMAT(o.order_date, '%Y-%m')
ORDER BY month;
```

RESULT

MONTH	TOTAL_CUSTOMERS	NEW_CUSTOMERS	RETURNING_CUSTOMERS
2026-01	19	19	0
2026-02	17	3	14
2026-03	16	1	15
2026-04	21	1	20
2026-05	21	1	20
2026-06	20	0	20
2026-07	18	0	18
2026-08	19	0	19
2026-09	17	0	17

WHAT IT TELLS YOU

Acquisition stopped dead after May: zero new customers in the final four months. Revenue held up only because the existing base kept buying. That is a business running on its back catalogue, and it is the kind of finding a revenue chart alone would never surface. The counting trick here is COUNT(DISTINCT CASE WHEN ...), which counts only rows matching the condition because CASE returns NULL otherwise and COUNT skips NULLs.

CUSTOMER BEHAVIOUR

Top Category per Customer

Problem: What is each customer's number one spending category?

```sql
WITH category_spend AS (
    SELECT c.id AS customer_id, c.full_name, c.segment, p.category,
        ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS spend,
        RANK() OVER (PARTITION BY c.id
            ORDER BY SUM(oi.quantity * oi.unit_price
                         * (1 - oi.discount)) DESC) AS category_rank
    FROM customers c
    JOIN orders o       ON c.id = o.customer_id
    JOIN order_items oi ON o.id = oi.order_id
    JOIN products p     ON oi.product_id = p.id
    WHERE o.current_status_id NOT IN (6)
    GROUP BY c.id, c.full_name, c.segment, p.category
)
SELECT full_name, segment, category AS top_category, spend
FROM category_spend
WHERE category_rank = 1
ORDER BY spend DESC;
```

RESULT (TOP 12 BY SPEND)

FULL_NAME	SEGMENT	TOP_CATEGORY	SPEND
Acme Corp	Corporate	Laptops	413,593.25
Vertex Systems	Corporate	Laptops	365,694.80
Apex Dynamics	Corporate	Laptops	309,086.25
GreenPath Inc	Corporate	Laptops	306,582.90
TechNova Ltd	Corporate	Laptops	184,952.35
DataBridge Co	Corporate	Laptops	93,699.30
Orbit Creative	SMB	Laptops	73,133.50
BlueSky LLC	SMB	Laptops	66,803.15
PineCrest Digital	SMB	Laptops	66,608.70
WaveLink Tech	SMB	Laptops	38,332.45
Crestwood Agency	SMB	Laptops	25,780.85
SilverEdge Media	SMB	Laptops	25,561.20

READ THIS RESULT CRITICALLY

Laptops is the top category for every single customer shown, which makes this a textbook case of a result that is technically correct and analytically useless. Because laptops carry the highest unit price, ranking by spend just re-discovers the price list. Rank by **units** or by share-of-wallet instead and the answer becomes informative.

CUSTOMER BEHAVIOUR

Basket Size Analysis

Problem: How many items are in a typical order, and how much is it worth?

```sql
SELECT c.full_name, c.segment,
    COUNT(DISTINCT o.id)                 AS orders,
    ROUND(AVG(basket.items_in_order), 1)  AS avg_items_per_order,
    ROUND(AVG(basket.order_value), 2)     AS avg_order_value,
    ROUND(MAX(basket.order_value), 2)     AS largest_order,
    ROUND(MIN(basket.order_value), 2)     AS smallest_order
FROM customers c
JOIN orders o ON c.id = o.customer_id
JOIN (
    SELECT order_id,
           COUNT(*)                                    AS items_in_order,
           SUM(quantity * unit_price * (1 - discount)) AS order_value
    FROM order_items GROUP BY order_id
) basket ON o.id = basket.order_id
WHERE o.current_status_id NOT IN (6)
GROUP BY c.id, c.full_name, c.segment
ORDER BY avg_order_value DESC;
```

RESULT (TOP 12 BY AVERAGE ORDER VALUE)

FULL_NAME	SEGMENT	ORDERS	AVG_ITEMS	AVG_VALUE	LARGEST	SMALLEST
Apex Dynamics	Corporate	23	4.1	19,391.00	37,072.15	8,120.40
Acme Corp	Corporate	39	3.8	15,474.89	35,415.70	3,931.95
Vertex Systems	Corporate	40	4.1	15,227.41	37,723.60	3,583.65
TechNova Ltd	Corporate	19	3.8	14,922.92	29,522.40	3,958.75
GreenPath Inc	Corporate	38	4.1	14,482.31	30,733.20	3,421.50
DataBridge Co	Corporate	17	3.9	10,373.80	20,355.75	3,885.75
Crestwood Agency	SMB	5	3.4	9,453.58	16,483.80	3,740.50
Orbit Creative	SMB	15	2.7	7,788.78	18,253.85	2,462.40
PineCrest Digital	SMB	19	3.2	6,975.07	16,423.95	703.75
BlueSky LLC	SMB	19	3.1	5,986.28	15,134.10	420.65
WaveLink Tech	SMB	18	3.0	4,664.59	15,522.75	706.80
SilverEdge Media	SMB	12	2.6	4,369.55	11,322.00	920.05

WHAT IT TELLS YOU

Basket *count* barely moves between segments, 4.1 items for Corporate against 2.6 to 3.2 for SMB, but basket *value* moves by 4x. Corporate customers are not buying many more things, they are buying more expensive things. Upselling here means trading up, not adding items.

CUSTOMER BEHAVIOUR

Discount Sensitivity

Problem: Do bigger discounts actually produce bigger orders?

```sql
WITH order_disc AS (
    SELECT o.id AS order_id, c.segment,
           AVG(oi.discount)                                  AS avg_disc,
           SUM(oi.quantity * oi.unit_price * (1 - oi.discount)) AS order_value
    FROM customers c
    JOIN orders o       ON c.id = o.customer_id
    JOIN order_items oi ON o.id = oi.order_id
    WHERE o.current_status_id NOT IN (6)
    GROUP BY o.id, c.segment
)
SELECT segment,
    CASE WHEN avg_disc = 0    THEN 'No Discount'
         WHEN avg_disc < 0.10 THEN 'Low (1-9%)'
         WHEN avg_disc < 0.20 THEN 'Mid (10-19%)'
         ELSE 'High (20%+)' END  AS discount_tier,
    COUNT(*)                     AS orders,
    ROUND(AVG(order_value), 2)   AS avg_order_value,
    ROUND(SUM(order_value), 2)   AS total_revenue
FROM order_disc
GROUP BY segment, discount_tier
ORDER BY segment, avg_order_value DESC;
```

RESULT

SEGMENT	DISCOUNT_TIER	ORDERS	AVG_ORDER_VALUE	TOTAL_REVENUE
Consumer	Low (1-9%)	80	2,281.54	182,523.55
Consumer	No Discount	26	1,893.92	49,242.00
Corporate	Low (1-9%)	16	17,330.13	277,282.10
Corporate	Mid (10-19%)	160	14,947.16	2,391,546.05
SMB	Low (1-9%)	97	6,022.47	584,179.20
SMB	Mid (10-19%)	2	4,696.20	9,392.40
SMB	No Discount	3	4,479.67	13,439.00

WHAT IT TELLS YOU

Corporate orders at a *low* discount average 17,330 while those at a mid discount average 14,947. The deeper discount is attached to the smaller order, so discounting is not buying larger baskets here, it is just giving away margin. Note the original version of this query grouped by an aggregate expression, which MySQL rejects; computing the tier per order in a CTE first is what makes it legal.

CUSTOMER BEHAVIOUR

Order Pattern by Day of Week

Problem: Which days do customers actually buy on?

```sql
SELECT DAYNAME(o.order_date)       AS day_of_week,
       DAYOFWEEK(o.order_date)     AS day_num,
       COUNT(DISTINCT o.id)        AS total_orders,
       COUNT(DISTINCT o.customer_id) AS unique_customers,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue
FROM orders o
JOIN order_items oi ON o.id = oi.order_id
WHERE o.current_status_id NOT IN (6)
GROUP BY DAYNAME(o.order_date), DAYOFWEEK(o.order_date)
ORDER BY day_num;
```

RESULT

DAY_OF_WEEK	DAY_NUM	TOTAL_ORDERS	UNIQUE_CUSTOMERS	REVENUE
Sunday	1	63	22	494,073.55
Monday	2	58	21	621,390.40
Tuesday	3	55	20	551,799.25
Wednesday	4	52	21	462,405.05
Thursday	5	58	19	497,554.75
Friday	6	55	20	514,444.35
Saturday	7	43	19	365,936.95

WHAT IT TELLS YOU

Sunday has the most orders (63) but Monday makes the most money (621,390 from 58 orders). Saturday is weakest on both counts. Grouping by DAYOFWEEK as well as DAYNAME is what keeps the rows in calendar order; sorting on the name alone would give you Friday, Monday, Saturday, alphabetically.

CUSTOMER BEHAVIOUR

Cohort Retention

Problem: Of the customers who first ordered in a given month, how many were still buying months later?

```sql
WITH cohorts AS (
    SELECT customer_id, DATE_FORMAT(MIN(order_date), '%Y-%m') AS cohort_month
    FROM orders WHERE current_status_id NOT IN (6) GROUP BY customer_id
),
cohort_orders AS (
    SELECT c.cohort_month,
           DATE_FORMAT(o.order_date, '%Y-%m') AS order_month,
           COUNT(DISTINCT o.customer_id)      AS active_customers
    FROM cohorts c
    JOIN orders o ON c.customer_id = o.customer_id
    WHERE o.current_status_id NOT IN (6)
    GROUP BY c.cohort_month, DATE_FORMAT(o.order_date, '%Y-%m')
),
cohort_sizes AS (
    SELECT cohort_month, COUNT(*) AS cohort_size FROM cohorts GROUP BY cohort_month
)
SELECT co.cohort_month, cs.cohort_size, co.order_month, co.active_customers,
       ROUND(co.active_customers * 100.0 / cs.cohort_size, 1) AS retention_pct
FROM cohort_orders co
JOIN cohort_sizes cs ON co.cohort_month = cs.cohort_month
ORDER BY co.cohort_month, co.order_month;
```

RESULT (JANUARY COHORT, PLUS START OF FEBRUARY)

COHORT_MONTH	COHORT_SIZE	ORDER_MONTH	ACTIVE_CUSTOMERS	RETENTION_PCT
2026-01	19	2026-01	19	100.0
2026-01	19	2026-02	14	73.7
2026-01	19	2026-03	14	73.7
2026-01	19	2026-04	16	84.2
2026-01	19	2026-05	16	84.2
2026-01	19	2026-06	15	78.9
2026-01	19	2026-07	14	73.7
2026-01	19	2026-08	14	73.7
2026-01	19	2026-09	12	63.2
2026-02	3	2026-02	3	100.0
2026-02	3	2026-03	1	33.3
2026-02	3	2026-04	3	100.0

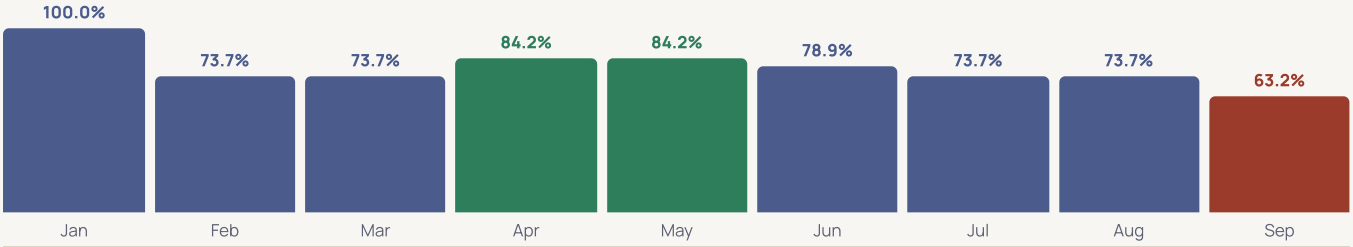
The January cohort is 19 of the 25 customers, so it dominates the analysis.

CUSTOMER BEHAVIOUR

How Cohort Analysis Works

1. **Assign every customer to a cohort.** The `cohorts` CTE takes each customer's `MIN(order_date)` and formats it to a month. A customer belongs to exactly one cohort forever, set by when they arrived.
2. **Count activity per cohort per month.** `cohort_orders` joins customers back to all of their orders and counts distinct actives in each month. One row per cohort per month.
3. **Get the denominator.** `cohort_sizes` counts how many customers each cohort started with. Without this the percentages have nothing to divide by.
4. **Divide.** actives in month M divided by original cohort size gives retention. The cohort's own first month is always 100% by construction, which is a useful correctness check.

Reading the January cohort



Retention of the 19-customer January cohort, as a percentage of its original size. Bars are scaled from zero. Green marks the April and May recovery, red the September low.

WHAT THE SHAPE TELLS YOU

Retention drops to 73.7% by month two, recovers to 84.2% in April and May, then slides to 63.2% by September. The recovery matters: a pure decay curve means people leave and never return, whereas a dip-and-rebound means customers buy in bursts. Treating a quiet month as churn would have been wrong for those customers in April.

WATCH THE DENOMINATOR

The February cohort is 3 customers, so one returning buyer swings retention by 33 points and it reads 100%, 33%, 100%. Always show cohort_size next to the percentage; a retention figure without its base is meaningless.

CUSTOMER BEHAVIOUR

Churn Risk Detection

Problem: Which customers have gone quiet relative to their own buying rhythm?

```sql
WITH asof AS (SELECT MAX(order_date) AS today FROM orders),
gaps AS (
    SELECT o.customer_id, o.order_date,
           DATEDIFF(o.order_date, LAG(o.order_date) OVER (
               PARTITION BY o.customer_id ORDER BY o.order_date)) AS gap_days
    FROM orders o WHERE o.current_status_id NOT IN (6)
),
customer_stats AS (
    SELECT c.id, c.full_name, c.segment,
           COUNT(DISTINCT g.order_date) AS total_orders,
           MAX(g.order_date)            AS last_order_date,
           DATEDIFF((SELECT today FROM asof), MAX(g.order_date)) AS days_since_last,
           ROUND(AVG(g.gap_days), 0)    AS avg_gap_days
    FROM gaps g JOIN customers c ON g.customer_id = c.id
    GROUP BY c.id, c.full_name, c.segment
)
SELECT full_name, segment, total_orders, last_order_date,
       days_since_last, avg_gap_days,
       CASE WHEN days_since_last > avg_gap_days * 2.5 THEN 'High Risk'
            WHEN days_since_last > avg_gap_days * 1.5 THEN 'Medium Risk'
            ELSE 'Active' END AS churn_risk
FROM customer_stats WHERE total_orders >= 2
ORDER BY days_since_last DESC;
```

RESULT (12 QUIETEST CUSTOMERS)

FULL_NAME	SEGMENT	ORDERS	LAST_ORDER	DAYS_SINCE	AVG_GAP	CHURN_RISK
Priya Mehta	Consumer	13	2026-06-22	99	13	High Risk
Liam Foster	Consumer	6	2026-07-12	79	35	Medium Risk
Noah Williams	Consumer	4	2026-07-14	77	59	Active
Sara Kim	Consumer	11	2026-08-19	41	19	Medium Risk
Tom Reynolds	Consumer	10	2026-08-23	37	21	Medium Risk
Redstone Partners	SMB	14	2026-08-24	36	18	Medium Risk
Crestwood Agency	SMB	5	2026-08-24	36	57	Active
Nina Patel	Consumer	10	2026-08-30	30	26	Active
Vertex Systems	Corporate	38	2026-09-08	21	6	High Risk
Chris Nguyen	Consumer	11	2026-09-11	18	25	Active
Orbit Creative	SMB	14	2026-09-14	15	13	Active
Zoe Campbell	Consumer	7	2026-09-15	14	13	Active

CUSTOMER BEHAVIOUR

Why Relative Churn Beats a Fixed Threshold

1. **Measure each customer's own rhythm.** The `gaps` CTE uses LAG to get the days between consecutive orders, then averages them per customer. This is the personal baseline.
2. **Measure silence.** `days_since_last` is the distance from the as-of date to their most recent order.
3. **Compare the two.** Silence longer than 2.5x their normal gap is High Risk; past 1.5x is Medium. The multiplier is the judgement call, and it belongs to the business, not the query.

A FIXED 60-DAY RULE WOULD SAY

Vertex Systems: **fine** (21 days)
Noah Williams: **at risk** (77 days)

THE RELATIVE RULE SAYS

Vertex Systems: **High Risk** (21 vs 6-day habit)
Noah Williams: **Active** (77 vs 59-day habit)

THE FINDING THAT PAYS FOR THE WHOLE QUERY

Vertex Systems is the single largest customer in the business at 609,096 lifetime revenue. It normally orders every **6 days** and has now been silent for **21**, three and a half times its habit. A fixed 60-day threshold would not flag it for another five weeks. Noah Williams, meanwhile, is a naturally slow 59-day buyer who a fixed rule would wrongly chase.

WHERE JUDGEMENT ENTERS

2.5x and 1.5x are chosen, not derived. Tune them against actual win-back outcomes, and segment them: a 6-day corporate cadence and a 59-day consumer cadence probably do not deserve the same multiplier.

CUSTOMER BEHAVIOUR

First Product Purchased

Problem: What do customers buy on their very first order? This is the entry point into the catalogue.

```sql
WITH first_orders AS (
    SELECT o.customer_id, o.id AS order_id, o.order_date,
           RANK() OVER (PARTITION BY o.customer_id
                        ORDER BY o.order_date) AS order_rank
    FROM orders o WHERE o.current_status_id NOT IN (6)
)
SELECT p.category, p.product_name,
       COUNT(*)                                          AS first_purchase_count,
       ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 1) AS pct_of_first_purchases
FROM first_orders fo
JOIN order_items oi ON fo.order_id = oi.order_id
JOIN products p     ON oi.product_id = p.id
WHERE fo.order_rank = 1
GROUP BY p.category, p.product_name
ORDER BY first_purchase_count DESC;
```

RESULT (TOP 12 ENTRY PRODUCTS)

CATEGORY	PRODUCT_NAME	FIRST_PURCHASE_COUNT	PCT_OF_FIRST
Laptops	Dell XPS 15	9	12.2
Monitors	Samsung 49 Ultrawide	7	9.5
Accessories	Webcam Logitech 4K	7	9.5
Accessories	Anker USB-C Hub	6	8.1
Laptops	ASUS ROG Zephyrus G14	6	8.1
Accessories	Elgato Stream Deck	6	8.1
Accessories	CalDigit TS4 Dock	5	6.8
Accessories	Logitech MX Master 3	5	6.8
Accessories	Keychron K2 Keyboard	5	6.8
Monitors	LG 27 4K Monitor	4	5.4
Laptops	MacBook Pro 14	3	4.1
Tablets	Microsoft Surface Pro 10	2	2.7

WHAT IT TELLS YOU

Dell XPS 15 is the most common entry product at 12.2%, despite MacBook Pro 14 being the revenue leader overall. Customers arrive on the cheaper laptop and trade up later. Note the counts exceed 25 because a first order can contain several products, so each of its lines is counted.

CUSTOMER BEHAVIOUR

Second-Order Category Path

Problem: After their first order, what do customers buy next?

```sql
WITH ranked_orders AS (
    SELECT o.customer_id, o.id AS order_id,
           ROW_NUMBER() OVER (PARTITION BY o.customer_id
                              ORDER BY o.order_date) AS order_num
    FROM orders o WHERE o.current_status_id NOT IN (6)
),
order_categories AS (
    SELECT ro.customer_id, ro.order_num,
           GROUP_CONCAT(DISTINCT p.category ORDER BY p.category) AS categories
    FROM ranked_orders ro
    JOIN order_items oi ON ro.order_id = oi.order_id
    JOIN products p     ON oi.product_id = p.id
    GROUP BY ro.customer_id, ro.order_num
)
SELECT a.categories AS first_order_category,
       b.categories AS second_order_category,
       COUNT(*)     AS customers_with_this_path
FROM order_categories a
JOIN order_categories b
    ON a.customer_id = b.customer_id
   AND a.order_num = 1 AND b.order_num = 2
GROUP BY a.categories, b.categories
ORDER BY customers_with_this_path DESC;
```

RESULT (TOP 8 PATHS)

FIRST_ORDER_CATEGORY	SECOND_ORDER_CATEGORY	CUSTOMERS
Accessories,Laptops,Monitors	Accessories,Laptops	2
Accessories,Monitors	Accessories,Laptops,Monitors	1
Laptops,Monitors	Accessories,Laptops,Monitors	1
Accessories,Laptops	Accessories,Laptops	1
Accessories,Laptops,Monitors	Accessories	1
Accessories,Laptops	Laptops,Monitors,Tablets	1
Accessories,Tablets	Accessories,Laptops	1
Laptops,Monitors,Tablets	Accessories,Laptops	1

KNOW WHEN A RESULT IS TOO THIN

Every path but one has a single customer behind it, so there is no pattern here worth acting on. With 25 customers and multi-category baskets, GROUP_CONCAT produces almost as many distinct combinations as customers. The technique is sound and would work on 25,000 customers; the honest read at this scale is that the sample is too small. Reporting a 1-customer path as a trend would be the real error.

CHAPTER 2 REFERENCE

Behaviour Patterns Cheat Sheet

QUESTION	PATTERN	KEY CLAUSE
Gap between events	LAG partitioned by customer	`DATEDIFF(d, LAG(d) OVER (PARTITION BY cust ORDER BY d))`
First / last event	MIN or MAX of the date	`MIN(order_date)` per customer
Nth event per customer	ROW_NUMBER partitioned	`ORDER BY order_date` then filter `= 1`
Bucket into named tiers	CASE in a subquery, group outside	aggregate first, label second
Count only matching rows	COUNT of a CASE	`COUNT(DISTINCT CASE WHEN c THEN id END)`
Share of total	Bare window, no ORDER BY	`SUM(COUNT(*)) OVER ()`
Cohort retention	Three CTEs: assign, activity, size	join actives to cohort size
Per-order metric	Derived table keyed on order_id	collapse before you average
Recency on a fixed extract	As-of date from the data	`(SELECT MAX(order_date) FROM orders)`
Relative anomaly	Compare value to its own baseline	`silence > avg_gap * 2.5`

THE HABIT THAT SEPARATES ANALYSTS

Three results in this chapter were technically correct and analytically worthless: top category returning Laptops for everyone, the 1-customer path table, and the 3-person February cohort reading 100%. Running the query is the easy half. Asking whether the answer means anything is the job.

CHAPTER 3

Operational Analytics

Revenue queries read the orders table. Operations queries read the **status log**, and that is where the interesting problems live: where orders stall, why they come back, and how fast the team moves them.

```
-- the order lifecycle, as recorded in order_status_log
1 Pending → 2 Confirmed → 3 Processing → 4 Shipped → 5 Delivered
exits: 6 Cancelled  |  after delivery: 7 Return Requested → 8 Returned → 9 Refunded
```

`orders`
Holds only the *current* status. Good for "where is it now", useless for history.

`order_status_log`
One row per transition with a timestamp. 2,148 rows across 432 orders. This is the operational record.

THE CORE OPERATIONAL TECHNIQUE

To measure time spent in a status you need consecutive log rows *paired up*. Every timing query in this chapter self-joins the log to itself, matching each row to the next one for the same order. Get that pairing right and the rest is arithmetic.

OPERATIONS

Order Fulfillment Time

Problem: How long does an order take from placement to delivery, month by month?

```sql
SELECT
    YEAR(o.order_date)  AS yr,
    MONTH(o.order_date) AS mth,
    COUNT(*)            AS delivered_orders,
    ROUND(AVG(DATEDIFF(o.delivery_date, o.order_date)), 1) AS avg_days_to_deliver,
    MIN(DATEDIFF(o.delivery_date, o.order_date))           AS min_days,
    MAX(DATEDIFF(o.delivery_date, o.order_date))           AS max_days
FROM orders o
WHERE o.current_status_id = 5          -- Delivered only
  AND o.delivery_date IS NOT NULL
GROUP BY YEAR(o.order_date), MONTH(o.order_date)
ORDER BY yr, mth;
```

RESULT

YR	MTH	DELIVERED_ORDERS	AVG_DAYS_TO_DELIVER	MIN_DAYS	MAX_DAYS
2026	1	35	5.2	3	7
2026	2	38	5.5	3	8
2026	3	34	5.5	3	8
2026	4	42	5.3	3	8
2026	5	43	6.2	3	8
2026	6	41	5.3	3	8
2026	7	38	5.3	3	8
2026	8	38	5.6	3	8
2026	9	22	5.2	3	8

WHAT IT TELLS YOU

Delivery is remarkably stable: 5.2 to 6.2 days every month, floor of 3 and ceiling of 8 throughout. May is the only soft month at 6.2. September shows 22 delivered rather than ~40 because orders placed late in the month have not finished the pipeline yet, not because volume collapsed. Always check whether a trailing period is simply incomplete before calling it a decline.

OPERATIONS

Time in Each Status

Problem: Where do orders actually spend their time between placement and delivery?

```sql
WITH log_pairs AS (
    SELECT a.order_id,
           sa.status_name AS from_name,
           sb.status_name AS to_name,
           TIMESTAMPDIFF(HOUR, a.changed_at, b.changed_at) AS hours_in_status
    FROM order_status_log a
    JOIN order_status_log b
        ON a.order_id = b.order_id
       AND b.id = (SELECT MIN(id) FROM order_status_log
                   WHERE order_id = a.order_id AND id > a.id)
    JOIN statuses sa ON a.status_id = sa.id
    JOIN statuses sb ON b.status_id = sb.id
)
SELECT from_name AS from_status, to_name AS to_status,
       COUNT(*)                      AS transitions,
       ROUND(AVG(hours_in_status), 1) AS avg_hours,
       MIN(hours_in_status)          AS min_hours,
       MAX(hours_in_status)          AS max_hours
FROM log_pairs
GROUP BY from_name, to_name
ORDER BY transitions DESC;
```

RESULT (ALL 8 OBSERVED TRANSITIONS)

FROM_STATUS	TO_STATUS	TRANSITIONS	AVG_HOURS	MIN_HOURS	MAX_HOURS
Pending	Confirmed	432	1.5	1	2
Confirmed	Processing	384	5.5	3	8
Processing	Shipped	381	37.9	2	76
Shipped	Delivered	366	84.0	41	129
Confirmed	Cancelled	48	8.8	1	18
Delivered	Return Requested	35	198.9	72	336
Return Requested	Returned	35	104.9	72	144
Returned	Refunded	35	50.1	24	72

WHAT IT TELLS YOU

The bottleneck is unambiguous. Pending to Confirmed takes 1.5 hours and Confirmed to Processing 5.5, but **Processing to Shipped averages 37.9 hours and peaks at 76**. That single step is the warehouse queue, and it is where a day and a half of every order's life disappears. Shipped to Delivered is longer at 84 hours but that is the carrier, largely outside your control.

OPERATIONS

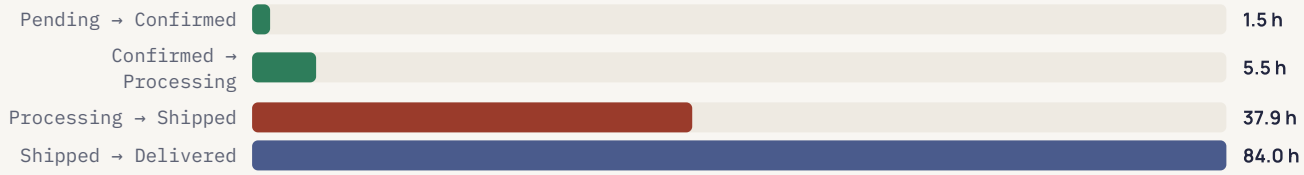
How the Status-Pairing Self-Join Works

The whole query rests on one correlated subquery. It is worth understanding properly, because this shape appears any time you need *consecutive* rows from a log.

```
JOIN order_status_log b
    ON a.order_id = b.order_id
   AND b.id = (SELECT MIN(id) FROM order_status_log
               WHERE order_id = a.order_id AND id > a.id)
```

1. **a.order_id = b.order_id** keeps the pairing inside one order. Without it you would match transitions across unrelated orders.
2. **id > a.id** restricts to rows that come after the current one.
3. **MIN(...)** of those takes the *very next* row, not all later rows. Drop the MIN and each row pairs with every subsequent row, inflating the counts enormously.
4. **TIMESTAMPDIFF(HOUR, a, b)** then measures the dwell time in the status the order was leaving.

Where the time actually goes



Average hours per step, scaled to the slowest. Red marks the internal bottleneck; blue is carrier time.

SEPARATE WHAT YOU CONTROL FROM WHAT YOU DO NOT

Of roughly 129 hours from order to delivery, 84 sit with the carrier and 45 are internal. The realistic target is the 37.9-hour Processing to Shipped step: halving it removes almost a full day from every order, with no carrier renegotiation required.

A FASTER ALTERNATIVE

On MySQL 8 you can replace the correlated subquery with `LEAD(changed_at) OVER (PARTITION BY order_id ORDER BY id)`, which gets the next timestamp in one pass instead of a subquery per row. Same answer, considerably faster on a large log.

OPERATIONS

Order Status Funnel

Problem: How many orders reach each stage of the lifecycle, and where do they fall out?

```sql
SELECT s.status_name,
       COUNT(DISTINCT osl.order_id) AS orders_reached,
       ROUND(COUNT(DISTINCT osl.order_id) * 100.0
             / (SELECT COUNT(*) FROM orders), 1) AS pct_of_all_orders
FROM statuses s
LEFT JOIN order_status_log osl ON s.id = osl.status_id
GROUP BY s.id, s.status_name
ORDER BY s.id;
```

RESULT

STATUS_NAME	ORDERS_REACHED	PCT_OF_ALL_ORDERS
Pending	432	100.0
Confirmed	432	100.0
Processing	384	88.9
Shipped	381	88.2
Delivered	366	84.7
Cancelled	48	11.1
Return Requested	35	8.1
Returned	35	8.1
Refunded	35	8.1

WHAT IT TELLS YOU

Every order confirms, then 48 (11.1%) drop out at Confirmed to Processing, which matches the Cancelled count exactly. From there the funnel is tight: 384 to 381 to 366. The LEFT JOIN matters, since a status nobody reached would still appear with a zero rather than vanishing from the report. Return Requested, Returned and Refunded all show 35, meaning every return request completed the full refund path with none abandoned midway.

OPERATIONS

Cancellation Rate by Month

Problem: What share of orders get cancelled, and is it getting better or worse?

```sql
SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS month,
       COUNT(*)                           AS total_orders,
       SUM(CASE WHEN o.current_status_id = 6 THEN 1 ELSE 0 END) AS cancelled,
       ROUND(SUM(CASE WHEN o.current_status_id = 6 THEN 1 ELSE 0 END) * 100.0
             / COUNT(*), 1)               AS cancellation_rate_pct
FROM orders o
GROUP BY DATE_FORMAT(o.order_date, '%Y-%m')
ORDER BY month;
```

RESULT

MONTH	TOTAL_ORDERS	CANCELLED	CANCELLATION_RATE_PCT
2026-01	46	6	13.0
2026-02	46	6	13.0
2026-03	50	12	24.0
2026-04	48	3	6.3
2026-05	52	5	9.6
2026-06	50	3	6.0
2026-07	48	2	4.2
2026-08	47	6	12.8
2026-09	45	5	11.1

WHAT IT TELLS YOU

March is the outlier at 24%, double any neighbouring month, and worth a root-cause investigation on its own. The trend improves to a 4.2% low in July then worsens again through August and September. Note this query deliberately omits the status filter used elsewhere: you cannot compute a cancellation rate from a dataset that has already excluded cancellations.

OPERATIONS

Return Rate by Category

Problem: Which product categories come back most often?

```sql
SELECT p.category,
    COUNT(DISTINCT CASE WHEN o.current_status_id = 5
                        THEN o.id END) AS delivered,
    COUNT(DISTINCT CASE WHEN o.current_status_id IN (8,9)
                        THEN o.id END) AS returned,
    ROUND(COUNT(DISTINCT CASE WHEN o.current_status_id IN (8,9) THEN o.id END)
          * 100.0 / NULLIF(COUNT(DISTINCT CASE WHEN o.current_status_id IN (5,8,9)
                                               THEN o.id END), 0), 1) AS return_rate_pct
FROM products p
JOIN order_items oi ON p.id = oi.product_id
JOIN orders o       ON oi.order_id = o.id
WHERE o.current_status_id IN (5, 8, 9)
GROUP BY p.category
ORDER BY return_rate_pct DESC;
```

RESULT

CATEGORY	DELIVERED	RETURNED	RETURN_RATE_PCT
Tablets	59	8	11.9
Laptops	177	21	10.6
Accessories	270	26	8.8
Audio	34	3	8.1
Monitors	185	16	8.0
Phones	24	1	4.0

WHAT IT TELLS YOU

Tablets and Laptops return at 11.9% and 10.6%, roughly triple the Phones rate of 4.0%. Laptops are the worst case commercially: highest revenue, thinnest margin at 19.1%, and the second-highest return rate. A returned laptop wipes out the margin on several successful ones.

OPERATIONS

Why Customers Return Things

Problem: What reasons are recorded when a return is requested?

```sql
SELECT osl.notes AS return_reason,
       COUNT(*)    AS occurrences,
       ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 1) AS pct
FROM order_status_log osl
WHERE osl.status_id = 7          -- Return Requested
GROUP BY osl.notes
ORDER BY occurrences DESC;
```

RESULT (ALL 35 RETURN REQUESTS)

RETURN_REASON	OCCURRENCES	PCT
Changed mind after purchase	7	20.0
Item received damaged during shipping	7	20.0
Defective product reported by customer	6	17.1
Compatibility issue with existing setup	5	14.3
Product not as described on website	4	11.4
Performance below customer expectations	4	11.4
Wrong item delivered	2	5.7

WHAT IT TELLS YOU

Only 20% are genuine buyer's remorse. The other 80% are fixable operational failures: 20% damaged in transit, 17.1% defective, 14.3% compatibility, 11.4% misleading product pages and 5.7% picking errors. Packaging and product-page accuracy alone address over 30% of all returns.

OPERATIONS

Return Rate and Delivery Speed by Segment

Problem: Do some customer segments return more than others, and is delivery speed a factor?

```sql
SELECT c.segment,
    COUNT(DISTINCT o.id) AS total_orders,
    SUM(CASE WHEN o.current_status_id IN (7,8,9) THEN 1 ELSE 0 END) AS returns,
    ROUND(SUM(CASE WHEN o.current_status_id IN (7,8,9) THEN 1 ELSE 0 END)
          * 100.0 / COUNT(DISTINCT o.id), 1) AS return_rate_pct,
    ROUND(AVG(CASE WHEN o.current_status_id = 5
                   THEN DATEDIFF(o.delivery_date, o.order_date) END), 1) AS avg_delivery_days
FROM customers c
JOIN orders o ON c.id = o.customer_id
WHERE o.current_status_id NOT IN (1, 2, 3)    -- exclude still in progress
GROUP BY c.segment
ORDER BY return_rate_pct DESC;
```

RESULT

SEGMENT	TOTAL_ORDERS	RETURNS	RETURN_RATE_PCT	AVG_DELIVERY_DAYS
SMB	113	15	13.3	5.3
Consumer	115	8	7.0	5.5
Corporate	201	12	6.0	5.5

WHAT IT TELLS YOU

SMB returns at 13.3%, more than double the Corporate 6.0%, while delivery speed is effectively identical across all three at 5.3 to 5.5 days. So speed is not the driver. The CASE inside AVG is doing quiet work here: it returns NULL for non-delivered orders and AVG skips NULLs, so the average covers only delivered orders while the return rate still counts everything.

OPERATIONS

Shipping SLA Compliance

Problem: Against a 3-day shipping SLA, how are we performing and what is at risk?

```sql
WITH shipped AS (
    SELECT DATEDIFF(ship_date, order_date) AS days_to_ship
    FROM orders WHERE ship_date IS NOT NULL
)
SELECT days_to_ship,
       COUNT(*)                                          AS orders,
       ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 1) AS pct_of_shipped,
       CASE WHEN days_to_ship <= 2 THEN 'Within SLA'
            WHEN days_to_ship = 3  THEN 'At SLA limit'
            ELSE 'SLA breach' END                        AS sla_status
FROM shipped
GROUP BY days_to_ship
ORDER BY days_to_ship;
```

RESULT

DAYS_TO_SHIP	ORDERS	PCT_OF_SHIPPED	SLA_STATUS
1	124	32.5	Within SLA
2	136	35.7	Within SLA
3	121	31.8	At SLA limit

WHAT IT TELLS YOU

Zero breaches: every order ships within 3 days. But 31.8% land exactly *on* the limit, so roughly a third of orders have no margin at all. Any small disruption converts that block into breaches overnight. This page started life as an SLA breach report; the data had no breaches, so the honest move was to reframe the question as compliance rather than invent a threshold that would produce rows.

OPERATIONS

Live Orders Dashboard

Problem: What is in flight right now, how old is it, and who owns it?

```sql
WITH asof AS (SELECT MAX(order_date) AS today FROM orders)
SELECT o.id AS order_id, c.full_name AS customer, c.segment, o.order_date,
       DATEDIFF((SELECT today FROM asof), o.order_date) AS age_days,
       s.status_name AS current_status, e.full_name AS sales_rep,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS order_value
FROM orders o
JOIN customers c    ON o.customer_id = c.id
JOIN employees e    ON o.employee_id = e.id
JOIN statuses  s    ON o.current_status_id = s.id
JOIN order_items oi ON o.id = oi.order_id
WHERE o.current_status_id IN (1, 2, 3, 4)     -- not delivered or closed
GROUP BY o.id, c.full_name, c.segment, o.order_date, s.status_name, e.full_name
ORDER BY age_days DESC;
```

RESULT (11 OLDEST OPEN ORDERS)

ORDER_ID	CUSTOMER	SEGMENT	ORDER_DATE	AGE	STATUS	SALES_REP	VALUE
1426	PineCrest Digital	SMB	2026-09-03	26	Shipped	James Caldwell	9,798.45
1402	BlueSky LLC	SMB	2026-09-03	26	Shipped	Priya Sharma	8,298.40
1430	Apex Dynamics	Corporate	2026-09-06	23	Shipped	David Okonkwo	10,665.55
1412	Ethan Brooks	Consumer	2026-09-08	21	Shipped	Priya Sharma	549.00
1410	GreenPath Inc	Corporate	2026-09-10	19	Shipped	David Okonkwo	5,206.00
1423	Apex Dynamics	Corporate	2026-09-11	18	Shipped	David Okonkwo	25,028.80
1411	Apex Dynamics	Corporate	2026-09-11	18	Shipped	David Okonkwo	17,692.75
1405	Acme Corp	Corporate	2026-09-12	17	Shipped	James Caldwell	13,146.20
1409	Mia Torres	Consumer	2026-09-16	13	Shipped	Sarah Mitchell	13,701.20
1413	GreenPath Inc	Corporate	2026-09-16	13	Shipped	Lena Fischer	20,147.65
1428	WaveLink Tech	SMB	2026-09-20	9	Shipped	Aisha Rahman	1,228.35

Age is measured from the dataset's latest order date, not the real-world today.

WHAT IT TELLS YOU

Every open order is already Shipped, so nothing is stuck in the warehouse. But the two oldest have been in transit 26 days against an 84-hour average, which is a carrier escalation, not a fulfilment problem. Sorted by age, the highest-value exposure (25,028 for Apex Dynamics) surfaces immediately.

OPERATIONS

Employee Operational Speed

Problem: How quickly does each employee move orders through the pipeline?

```sql
SELECT e.full_name AS employee,
       COUNT(DISTINCT osl.order_id) AS status_updates_made,
       ROUND(AVG(TIMESTAMPDIFF(HOUR, prev_log.changed_at,
                               osl.changed_at)), 1) AS avg_hours_per_update,
       MIN(TIMESTAMPDIFF(HOUR, prev_log.changed_at, osl.changed_at)) AS fastest_hrs,
       MAX(TIMESTAMPDIFF(HOUR, prev_log.changed_at, osl.changed_at)) AS slowest_hrs
FROM order_status_log osl
JOIN employees e ON osl.changed_by_employee = e.id
JOIN order_status_log prev_log
    ON prev_log.order_id = osl.order_id
   AND prev_log.id = (SELECT MAX(id) FROM order_status_log
                      WHERE order_id = osl.order_id AND id < osl.id)
GROUP BY e.id, e.full_name
ORDER BY avg_hours_per_update ASC;
```

RESULT (ALL 7 EMPLOYEES)

EMPLOYEE	STATUS_UPDATES_MADE	AVG_HOURS_PER_UPDATE	FASTEST_HRS	SLOWEST_HRS
Lena Fischer	58	19.9	2	120
Sarah Mitchell	61	22.6	1	120
David Okonkwo	64	26.2	3	144
Marcus Webb	70	26.2	1	144
James Caldwell	62	26.9	2	144
Priya Sharma	66	27.4	3	144
Aisha Rahman	51	27.5	3	144

WHAT IT TELLS YOU

Note this uses `MAX(id) ... < osl.id`, the mirror image of the previous pairing query: it looks *backward* to the preceding row rather than forward. The spread is modest, 19.9 to 27.5 hours, so no one is a bottleneck. Read this cautiously though: the metric reflects which orders someone happened to handle as much as their personal speed, so it is a prompt for a conversation, not a performance score.

OPERATIONS

Repeat vs One-Time Buyers

Problem: How much more is a repeat customer worth than a one-time buyer?

```sql
SELECT buyer_type,
       COUNT(*)                    AS customers,
       ROUND(AVG(total_revenue), 2) AS avg_revenue,
       ROUND(AVG(order_count), 1)   AS avg_orders
FROM (
    SELECT c.id,
        COUNT(DISTINCT o.id) AS order_count,
        SUM(oi.quantity * oi.unit_price * (1 - oi.discount)) AS total_revenue,
        CASE WHEN COUNT(DISTINCT o.id) = 1  THEN 'One-Time'
             WHEN COUNT(DISTINCT o.id) <= 3 THEN 'Occasional'
             ELSE 'Repeat' END AS buyer_type
    FROM customers c
    JOIN orders o       ON c.id = o.customer_id
    JOIN order_items oi ON o.id = oi.order_id
    WHERE o.current_status_id NOT IN (6, 7, 8, 9)
    GROUP BY c.id
) buyer_summary
GROUP BY buyer_type
ORDER BY avg_revenue DESC;
```

RESULT

BUYER_TYPE	CUSTOMERS	AVG_REVENUE	AVG_ORDERS
Repeat	24	134,455.79	14.5
One-Time	1	663.10	1.0

WHAT IT TELLS YOU

A repeat customer is worth 134,455 against 663 for the single one-time buyer, a 200x gap, on 14.5 orders each. The Occasional bucket is absent entirely because no customer has 2 to 3 orders. With one customer in the One-Time bucket the ratio is not statistically meaningful, but the direction is unmistakable and consistent with the 84% loyal share from Chapter 2.

CHAPTER 3 REFERENCE

Operational Patterns Cheat Sheet

QUESTION	PATTERN	KEY CLAUSE
Next event in a log	Self-join on MIN(id) greater than current	`b.id = (SELECT MIN(id) ... AND id > a.id)`
Previous event in a log	Self-join on MAX(id) less than current	`prev.id = (SELECT MAX(id) ... AND id < osl.id)`
Faster alternative to both	LEAD / LAG over the log	`LEAD(changed_at) OVER (PARTITION BY order_id ORDER BY id)`
Duration between events	TIMESTAMPDIFF on the pair	`TIMESTAMPDIFF(HOUR, a.changed_at, b.changed_at)`
Funnel counts	LEFT JOIN from the dimension	keeps zero-count stages visible
Rate of a sub-population	Conditional SUM over total COUNT	`SUM(CASE WHEN c THEN 1 ELSE 0 END) / COUNT(*)`
Average over a subset only	CASE inside AVG returning NULL	AVG skips NULLs automatically
Distinct counts per condition	COUNT DISTINCT of a CASE	`COUNT(DISTINCT CASE WHEN c THEN id END)`
Age of open items	As-of date from the data	`DATEDIFF((SELECT MAX(order_date) ...), order_date)`
Banding a measure	CASE over a CTE column	compute the value first, band it second

THE FILTER RULE THAT CATCHES PEOPLE OUT

Revenue queries exclude statuses 6 to 9. Operational queries usually must not. A cancellation rate computed on data with cancellations already filtered out is always 0%. Ask what the denominator should be *before* copying a WHERE clause from another query.

OPERATIONAL HEADLINE FROM THIS CHAPTER

Processing to Shipped is the one internal step worth fixing (37.9 h average, 76 h worst). 80% of returns are operational rather than remorse. And 31.8% of orders ship exactly on the SLA limit with no slack at all.

REFERENCE

Choosing the Right Tool

IF YOU NEED TO	REACH FOR	BECAUSE
Compare a row to the previous one	`LAG()`	One pass, no self-join, NULL on the first row
Compare a row to its group's total	`SUM(x) OVER ()`	Keeps every row while adding the total
Accumulate through a series	`SUM() OVER (ORDER BY ... ROWS ...)`	The frame defines what accumulates
Pick the top N inside each group	`ROW_NUMBER()` + outer filter	Window functions cannot go in WHERE
Rank where ties should share a place	`RANK()` / `DENSE_RANK()`	RANK skips after a tie, DENSE_RANK does not
Name a sub-result and reuse it	CTE (`WITH`)	Readable, testable on its own
Aggregate then window over it	CTE, then the window outside	Windows run after aggregation, never inside
Count rows meeting a condition	`COUNT(CASE WHEN ... END)`	CASE yields NULL and COUNT skips NULLs
Average only part of the rows	`AVG(CASE WHEN ... END)`	Same NULL-skipping trick
Check existence, not values	`EXISTS` / `NOT EXISTS`	Stops at first match, safe with NULLs
Pair each row with the next	Self-join on `MIN(id) > current`	Or `LEAD()`, which is faster
Collapse to one row per entity	Derived table, then join	Prevents fan-out from double counting

THREE RULES THAT PREVENT MOST WRONG ANSWERS

Never average an average. Collapse to one row per order before averaging order value. **Never window inside an aggregate.** Aggregate in a CTE, window outside it. **Never reuse a WHERE clause without thinking.** The filter that makes a revenue query correct makes a cancellation query meaningless.

PRACTICE

10 Practice Questions

Each question gives the business problem and the exact result the correct query returns. Write the SQL yourself, then check your output row for row against the expected table.

Q1 MARGIN
Rank every product category by profit margin, not revenue.

Q2 TOP-N PER GROUP
Find the top 3 customers by revenue within each segment.

Q3 LAG
Identify the months with the largest revenue drops.

Q4 NOT EXISTS
List customers who have never had an order cancelled.

Q5 BELOW AVERAGE
Find products selling below their own category average.

Q6 RUNNING TOTAL
Show cumulative orders and a 3-month moving average.

Q7 EFFICIENCY
Rank sales reps by revenue per order, not total revenue.

Q8 CORRELATED
Find customers whose average order beats their segment average.

Q9 FUNNEL
Build the delivery funnel showing drop-off at each stage.

Q10 RANK
Rank days of the week by revenue and average order value.

HOW TO USE THESE

The expected results are real output, so if your query returns different numbers it is wrong, not merely different. Match the row count first, then the ordering, then the values. Hints appear under each question, and every technique you need has already appeared somewhere in the preceding chapters.

PRACTICE

Q1 MARGIN

Rank all six product categories by gross profit margin percentage, highest first. Show revenue and gross profit alongside.

EXPECTED RESULT

CATEGORY	REVENUE	GROSS_PROFIT	MARGIN	RANK
Audio	16,735.95	7,050.95	42.1	
Accessories	379,449.15	156,712.15	41.3	
Monitors	766,978.75	229,888.75	30.0	
Tablets	137,835.40	40,095.40	29.1	
Phones	38,664.00	11,124.00	28.8	
Laptops	1,887,938.80	361,298.80	19.1	

HINT

Margin is `SUM(qty * (price*(1-disc) - cost)) / SUM(qty * price*(1-disc))`. Put that whole expression inside `RANK() OVER (ORDER BY ... DESC)`.

Q2 TOP-N PER GROUP

For each customer segment, return the top 3 customers by net revenue, with their rank inside the segment.

EXPECTED RESULT

SEGMENT	FULL_NAME	REVENUE	RANK
Consumer	Priya Mehta	29,548.05	1
Consumer	Ethan Brooks	27,691.65	2
Consumer	Jake Morrison	27,130.55	3
Corporate	Vertex Systems	590,866.40	1
Corporate	Acme Corp	579,033.55	2
Corporate	GreenPath Inc	521,289.75	3
SMB	PineCrest Digital	131,822.55	1
SMB	BlueSky LLC	107,393.40	2
SMB	Orbit Creative	86,185.95	3

HINT

`ROW_NUMBER() OVER (PARTITION BY segment ORDER BY SUM(...) DESC)` inside a CTE, then `WHERE rn <= 3` outside it. You cannot filter a window function in the same SELECT that defines it.

PRACTICE

Q3 LAG

Find the months where revenue fell the most against the prior month. Return the four worst, sorted by percentage change ascending.

EXPECTED RESULT

MONTH	REVENUE	PREV_MONTH	CHANGE_PCT
2026-01	348,626.45	*NULL*	*NULL*
2026-02	309,886.80	348,626.45	-11.1
2026-08	324,594.20	355,937.40	-8.8
2026-07	355,937.40	377,738.10	-5.8

HINT

Aggregate monthly in a CTE, then LAG. January sorts first because **MySQL places NULLs ahead of all values in an ASC sort**. Add `WHERE change_pct IS NOT NULL` in an outer query if you want it gone.

Q4 NOT EXISTS

List every customer who has never had a single order cancelled, with their total order count.

EXPECTED RESULT

FULL_NAME	SEGMENT	TOTAL_ORDERS
BlueSky LLC	SMB	19
WaveLink Tech	SMB	18
Sara Kim	Consumer	12
Ethan Brooks	Consumer	12
Jake Morrison	Consumer	11
Mia Torres	Consumer	8
Zoe Campbell	Consumer	7
Ava Chen	Consumer	2

HINT

`WHERE NOT EXISTS (SELECT 1 FROM orders o2 WHERE o2.customer_id = c.id AND o2.current_status_id = 6)`. Prefer NOT EXISTS over NOT IN here: NOT IN returns nothing at all if the subquery yields a single NULL.

PRACTICE

Q5 BELOW AVERAGE

Find products whose revenue is below the average revenue of their own category. Show the shortfall, worst first.

EXPECTED RESULT

PRODUCT_NAME	CATEGORY	REVENUE	CAT_AVG	BEL
LG 27 4K Monitor	Monitors	137,714.85	255,659.58	-11
Dell 32 Curved Monitor	Monitors	175,981.95	255,659.58	-7
ASUS ROG Zephyrus G14	Laptops	567,134.75	629,312.93	-6
Dell XPS 15	Laptops	569,180.05	629,312.93	-6
Anker USB-C Hub	Accessories	19,761.70	63,241.53	-4
Logitech MX Master 3	Accessories	39,694.05	63,241.53	-2
Keychron K2 Keyboard	Accessories	39,959.40	63,241.53	-2
iPad Air	Tablets	31,832.50	45,945.13	-1
Samsung Tab S9	Tablets	38,584.80	45,945.13	-
iPhone 15 Pro	Phones	16,186.50	19,332.00	-

HINT

Use **two CTEs**: product revenue, then category averages, then join and filter. Doing it with a window average and filtering in WHERE gives wrong numbers, because windows are evaluated *after* WHERE, so the average you display is not the one you filtered on.

Q6 RUNNING TOTAL

Show orders per month with a cumulative running total and a 3-month moving average.

EXPECTED RESULT

MONTH	ORDERS	RUNNING	MOVING_AVG_3M
2026-01	46	46	46.0
2026-02	46	92	46.0
2026-03	50	142	47.3
2026-04	48	190	48.0
2026-05	52	242	50.0
2026-06	50	292	50.0
2026-07	48	340	50.0
2026-08	47	387	48.3
2026-09	45	432	46.7

HINT

Two different frames on the same ordered window: `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` accumulates, while `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` gives the rolling 3-month average.

PRACTICE

Q7 EFFICIENCY

Rank sales reps by revenue per order rather than total revenue. Does the leaderboard change?

EXPECTED RESULT

FULL_NAME	ORDERS	REVENUE	PER_ORDER	RK
Marcus Webb	54	552,834.55	10,237.68	1
Lena Fischer	50	498,343.80	9,966.88	2
James Caldwell	48	454,450.20	9,467.71	3
Priya Sharma	56	519,384.15	9,274.72	4
Aisha Rahman	41	373,967.25	9,121.15	5
David Okonkwo	50	444,604.30	8,892.09	6
Sarah Mitchell	50	384,017.80	7,680.36	7

HINT

Divide the revenue SUM by `COUNT(DISTINCT o.id)`, and rank on that ratio. Compare with the Chapter 1 leaderboard: Priya Sharma drops from 2nd to 4th, and Aisha Rahman climbs from 7th to 5th.

Q8 CORRELATED

Find customers whose average order value exceeds the average for their own segment. Top 10 by the gap.

EXPECTED RESULT

FULL_NAME	CUST_AOV	SEG_AOV	ABOVE_BY
Apex Dynamics	19,516.52	15,292.80	4,223.72
Crestwood Agency	7,979.54	5,783.36	2,196.18
Orbit Creative	7,835.09	5,783.36	2,051.73
PineCrest Digital	7,323.48	5,783.36	1,540.12
BlueSky LLC	6,317.26	5,783.36	533.90
Acme Corp	15,649.56	15,292.80	356.75
Ethan Brooks	2,517.42	2,208.48	308.95
Nina Patel	2,468.63	2,208.48	260.15
Jake Morrison	2,466.41	2,208.48	257.94
Vertex Systems	15,549.12	15,292.80	256.31

HINT

Collapse to one row per order first, or your averages will be weighted by line count. Build per-customer and per-segment averages as two CTEs, join on segment, filter where customer beats segment.

PRACTICE

Q9 FUNNEL

Build the delivery funnel for statuses 1 to 5, showing how many orders dropped out at each stage.

EXPECTED RESULT

STATUS	REACHED	PCT	DROPPED
Pending	432	100.0	*NULL*
Confirmed	432	100.0	0
Processing	384	88.9	48
Shipped	381	88.2	3
Delivered	366	84.7	15

HINT

Count distinct order_id per status from the log, then use `LAG(COUNT(DISTINCT osl.order_id)) OVER (ORDER BY s.id)` minus the current count to get the drop. The window sits outside the aggregate, which is legal because it operates on the grouped result.

Q10 RANK

Rank the days of the week by total revenue, and show average order value for each.

EXPECTED RESULT

DAY	ORDERS	REVENUE	AVG_VALUE	RANK
Monday	52	574,875.75	11,055.30	1
Tuesday	52	542,086.80	10,424.75	2
Thursday	54	471,177.75	8,725.51	3
Friday	50	469,904.75	9,398.10	4
Sunday	57	429,719.15	7,538.93	5
Wednesday	46	403,344.95	8,768.37	6
Saturday	38	336,492.90	8,855.08	7

HINT

Group by `DAYNAME(order_date)` and rank on the revenue SUM. Notice Sunday has the most orders (57) but ranks 5th on revenue, because its average order value is the lowest of the week. Counts differ slightly from the Chapter 2 version, which used a wider status filter.

FIVE FINDINGS THIS HANDBOOK SURFACED

Laptops are a trap. 58% of revenue at the catalogue's worst margin, 19.1%, and the second-highest return rate.

Acquisition has stopped. Zero new customers in the last four months; revenue is riding on the existing base.

The biggest customer is going quiet. Vertex Systems normally orders every 6 days and has been silent for 21.

One step owns the delay. Processing to Shipped averages 37.9 hours and peaks at 76.

Most returns are preventable. Only 20% are buyer's remorse; the rest are damage, defects and bad product pages.

None of that is visible in a revenue chart.

Every one came from joining orders to the status log, comparing a row to its own history, or asking whether a correct result actually meant anything. That last habit is the one worth practising.

More SQL and analytics content on LinkedIn

linkedin.com/in/sajibkhansk